

Foundations for programming languages

11: Logic programming

Isao Sasano

Logic programming

- Originated in automated theorem proving from which logic programming took the notion of a deduction
 - What is new in logic programming is that in the process of deduction some values are computed.
- Based on the syntax of first-order logic
- Prolog (1973) --- firstly used for natural language processing
- Robert Kowalski: “algorithm = logic + control”
- Alain Colmerauer and his team was developing a language for natural language processing and they collaborated with Kowalski and led to the development of Prolog.

Algorithm = logic + control

- Logic --- the facts and rules specifying what the algorithm does (programmers write)
- Control --- how the algorithm can be implemented (provided by languages)

Prolog has several dialects and each of them has their own control. Edinburgh Prolog is the de facto standard dialect and made a big influence on the ISO Prolog.

Non-syntactic differences between dialects can be illustrated by a family of equations:

$$\text{algorithm}_D = \text{logic} + \text{control}_D$$

(Reference) R. A. Kowalski, "Algorithm = Logic + Control".
Communication of the ACM, 22(7), pp. 424-436, 1979

Idea of logic programming

- Use relations instead of functions.
- Relation is a table with $n \geq 0$ columns and a possibly infinite set of rows.
 - A tuple $(a_1, a_2, \dots, a_n)$ is *in* a relation if the tuple appears in some row in the table for the relation.

An example: append

X	Y	Z
$[]$	$[]$	$[]$
$[a]$	$[]$	$[a]$
$[a]$	$[b]$	$[a,b]$
$[a,b]$	$[c,d]$	$[a,b,c,d]$
$\dots$	$\dots$	$\dots$

A relation *append* is a set of tuples of the form (X, Y, Z) where Z consists of the elements of X followed by the elements of Y . Relations are also called predicates because a relation R can be thought of a test of the form “Is a given tuple in relation R ?”

(ex.) $([a],[b],[a,b]) \sqsubseteq \text{append}$, $([a],[b],[]) \not\sqsubseteq \text{append}$

Horn clauses

Relations are described as rules of the form $P :- Q_1, Q_2, \dots, Q_k$. ($k \geq 0$), which corresponds to the logical expression:

$$P \Leftarrow Q_1 \wedge Q_2 \wedge \dots \wedge Q_k. \quad (k \geq 0)$$

This means that P holds when $Q_1, Q_2, \dots, Q_k$ hold (*declarative interpretation*). We consider this as follows: In order to establish (or deduce) P establish $Q_1, Q_2, \dots, Q_k$ (*procedural interpretation*). The rules are called *Horn clauses*. When $k=0$ the rule represents a *fact* and we do not write $:-$ and write just as P . .

(ex.) The relation `append` is described as the two rules.

`append ([ ], Y, Y).`

`append ( [H|X], Y, [H|Z] ) :- append (X,Y,Z).`

(Reference) A. Horn, "On sentences which are true of direct unions of algebras". *Journal of Symbolic Logic*, Vol. 16, pp. 14-21, 1951.

Examples of queries

?- **append**([a,b],[c,d],[a,b,c,d]).

yes

?- **append**([a,b],[c,d],Z).

Z=[a,b,c,d]

yes

?- **append**([a,b],Y,[a,b,c,d]).

Y=[c,d]

yes

?- **append**(X,[c,d],[a,b,c,d]).

X=[a,b]

yes

?- **append**(X,[d,c],[a,b,c,d]).

no

Computation in logic programming is to find answers for the given query, which may include variables.

Terms

A simple term:

- a number --- 0, 1972, etc.

- a variable, which must start with an uppercase letter
--- X, Source, etc.

- an atom (standing for itself) --- lisp, algol60, etc.

A compound term:

- an atom followed by a parenthesized sequence of subterms --- link(bcpl, c), path(L, M), etc.

In some cases a compound term may be written in infix.

(ex.) $=(X,Y)$ can be written as $X=Y$.

The special variable `_` is a placeholder for an unnamed term.

Syntax of facts, rules, and queries (in Edinburgh Prolog)

<fact> ::= <term> .

<rule> ::= <term> :- <terms> .

<query> ::= <terms> .

*<term> ::= <number> | <atom> | <variable>
 | <atom> (<terms>)*

<terms> ::= <term> | <term>, <terms>

Examples of facts and rules

link(fortran, algol60).

link(algol60, cpl).

link(cpl, bcpl).

link(bcpl, c).

link(c, cplusplus).

link(algol60, simula67).

link(simula67, cplusplus).

link(simula67, smalltalk80).

path(L,L).

path(L,M) :- link(L,X), path(X,M) .

Existential queries

A query $\langle term \rangle_1, \langle term \rangle_2, \dots, \langle term \rangle_k$ corresponds to the logical expression: “ $\exists X_1, \dots, \langle term \rangle_1 \wedge \langle term \rangle_2 \wedge \dots \wedge \langle term \rangle_k$ ”, where $X_1, \dots$ are variables in the terms.

Queries are also called *goals*. Terms in a query may be called as *subgoals*.

(ex.) ?- link(cpl, bcpl), link(bcpl, c).

yes

?- link(algol60, L), link(L, M).

L=cpl

M=bcpl

$\exists L. \exists M. \text{link}(\text{algol60}, L) \wedge \text{link}(L, M)?$
--

By typing return here, Prolog responds with yes to indicate that there might be more solutions and immediately prompts for the next query. By typing ;, Prolog responds with another solution or with no to indicate that no further solutions can be found.

(Cont.)

?- *link(algol60, L), link(L, M).*

L=cpl

M=bcpl ;

L=simula67

M=cplusplus ;

L=simula67

M=smalltalk80

yes

If Prolog found there are no more solutions it responds with *yes* without typing ;.

Universal facts and rules

A rule “ $\langle term \rangle \text{ :- } \langle term \rangle_1, \langle term \rangle_2, \dots, \langle term \rangle_k$.” corresponds to the logical expression “ $\forall X_1 \dots \langle term \rangle \Leftarrow \exists Y_1 \dots \langle term \rangle_1 \wedge \langle term \rangle_2 \dots \wedge \langle term \rangle_k$ ”, where $X_1 \dots$ are variables in the *head* term and $Y_1 \dots$ are the remaining variables in the *bodies*. The term to the left of the :- is called a *head* and the terms to the right of the :- is called *conditions* or *bodies*. A fact is a special case of rules and has a head and no conditions.

path(L,L).

path(L,M) :- link(L,X), path(X,M) .

defines a relation *path*. The fact “path(L,L)” represents that for every L, path(L,L) holds. The rule “path(L,M) :- link(L,X), path(X,M).” represents that for every L and M, path(L, M) holds when there exists X satisfying link(L,X) and path(X,M).

Negation as failure

Prolog answers *no* to a query if it fails to satisfy the query.

(ex.) ?- `link(lisp,scheme)` .

no

The not operator (not in Prolog) represents negation as failure rather than logical negation. A query $\text{not}(P)$ is true if the system fails to deduce P .

(Cont.)

?- *link(L,N), link(M,N) .*

L=fortran

M=fortran

N=algol60

?- *link(L,N), link(M,N), ✕+(L=M) .*

L=c

M=simula67

N=cplusplus ;

L=simula67

M=c

N=cplusplus ;

no

?- *✕+(L=M), link(L,N), link(M,N) .*

no

Unification

- Deduction in Prolog is based on the concept of unification; the two terms T_1 and T_2 *unify* if they have a common instance U . Unification is to obtain most general unifier for given two terms.
- Unification is applied for checking whether or not some rule can be applied to some term. Also, Prolog provides a built-in predicate for doing unification.

(ex.) ?- $f(X,b) = f(a,Y)$.

$X=a$

$Y=b$

(Reference1) John A. Robinson. “A machine-oriented logic based on the resolution principle”. *Journal of the ACM*, 12(1):23–41, 1965.

(Reference2) Alberto Martelli and Ugo Montanari, “An efficient unification algorithm”, *ACM TOPLAS* 4(2), pp. 258-282, 1982.

Instance

A term U is an instance of T , if $U = T\sigma$ for some substitution σ .

(ex1) The term $f(a,b)$ is an instance of the term $f(X,b)$.

(ex2) The term $f(a,b)$ is an instance of $f(a,Y)$.

(ex3) The term $g(a,a)$ is an instance of the term $g(X,X)$.

(ex4) The term $g(h(b),h(b))$ is an instance of the term $g(X,X)$.

(ex5) The term $g(a,b)$ is *not* an instance of the term $g(X,X)$.

We say terms T_1, T_2 *unify* if they have a same instance T .

(cf.) Unification is also used for type inference in implicitly typed functional languages.

Occurs check

- When we unify a variable X and a term T , checking whether or not X appears in T is said to be *occurs check*.

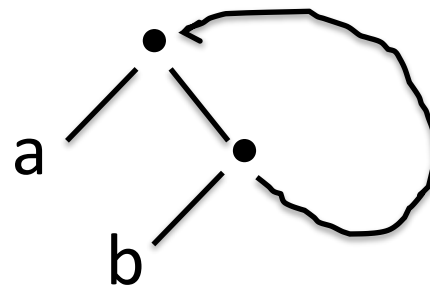
?- **append**([], E, [a,b|E]).

$E = [a,b,a,b,a,b,a,b,a,b,\dots]$

For `append`([], E, [a,b|E]) to unify with `append`([], Y, Y), Y must unify with the terms E and [a,b|E]. When we attempt to substitute [a,b|E] for E, we obtain

$E = [a,b|E] = [a,b,a,b|E] = [a,b,a,b,a,b|E] = \dots$

Some variants of Prolog like GNU Prolog construct cyclic terms in this kind of cases.



Arithmetic operators

- = operator --- $T_1 = T_2$ does unification of T_1 and T_2
- ?- $X = 2 + 3$.
 $X = 2 + 3$
- is operator --- T is e does unification of the result of evaluating the expression e and the term T .
- ?- X is $2 + 3$.
 $X = 5$
?- X is $2 + 3$, $X = 5$.
 $X = 5$
?- X is $2 + 3$, $X = 2 + 3$.
no

Prolog search trees

- Each node represents a goal.
- Each node has children, one for a rule that can be applied to the left most subgoal in the node. The order of children is the same as the order of the rules in the Prolog source code.
- Computation in Prolog proceeds by searching Prolog search tree in depth-first order. When it arrives at an empty node (i.e., a node with no goal), it responds with the solution. When it arrives at a non-empty node that has no children, it backtracks.

(Ex.)

a(1) :- b.

a(2) :- e.

b :- c.

b :- d.

c :- **fail**.

d.

e.

(**fail** is a predicate
that always fails)

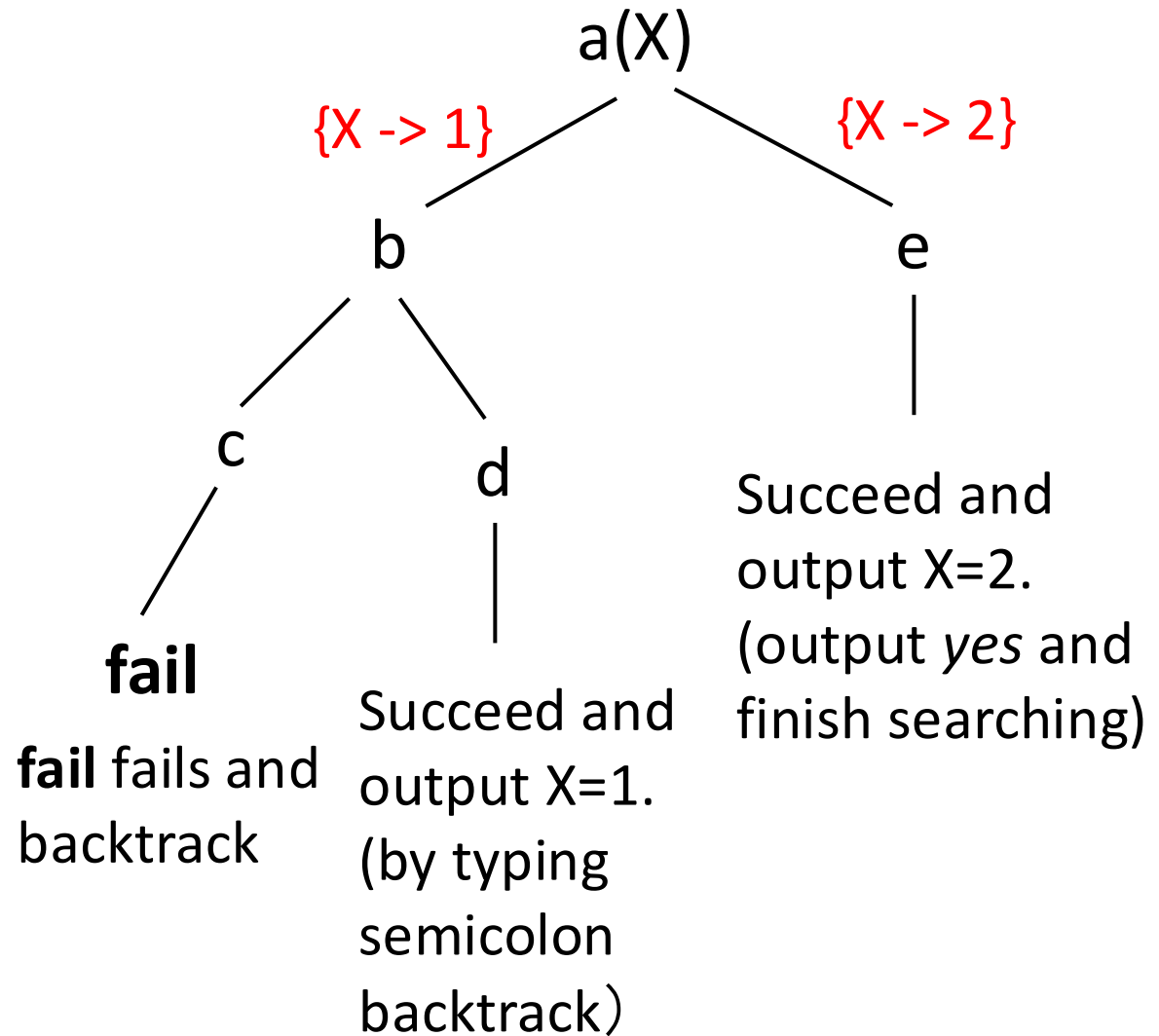
?- a(X).

X=1;

X=2

yes

Prolog search
tree



Cut !

- Cut ! cuts some portion of Prolog search tree and reduces time for computation.
- Terms having cut may not have the same meaning as the corresponding logical expression.

$B :- C_1, \dots, C_{j-1}, !, C_{j+1}, \dots, C_k$

The cut ! always succeeds and the predicate B fails when the control comes back to ! by backtracking.

a(1) :- b. ?- a(X).

a(2) :- e. X=1;

b :- c. X=2

b :- d. yes

c :- fail.

d.

e.

a(1) :- b. ?- a(X).

a(2) :- e. X=2

b :- !, c. yes

b :- d.

c :- fail.

d.

e.

An example of usage of cut

```
mem(K, node(K,_,_)).
```

```
mem(K, node(N,S,_)) :- K<N, mem(K,S).
```

```
mem(K, node(N,_,T)) :- K>N, mem(K,T).
```

In this definition of mem, the three rules are mutually exclusive.

```
mem(K, node(K,_,_)).
```

```
mem(K, node(N,S,_)) :- K<N, !, mem(K,S).
```

```
mem(K, node(N,_,T)) :- K>N, mem(K,T).
```

In this definition of mem, ! cuts the portion of Prolog search tree that does not have solutions. This kind of cuts are called *green cuts*. Others are called *red cuts*.

The not operator

[The not operator in Prolog]

$\backslash+(Y) :- Y, !, \text{fail}.$

$\backslash+(_).$

?- $X=2, \backslash+(X=1).$

$X=2$

yes

?- $\backslash+(X=1), X=2.$

no

